

AFIP — Aplicación de Finanzas Personales

Nombre del Proyecto: AFIP — Aplicación de Finanzas Personales

Fecha: 2025

Versión: 1.0

Contenido

1. Requisitos Previos

- a. Hardware: Servidor Ubuntu 22.04 LTS, 4+ cores, 8 GB RAM (recomendado), 50 GB SSD (producción). Mínimo: 2 cores, 2 GB RAM, 10 GB HDD.
- b. Software: Node.js 18.x o superior, npm 9.x o superior, MySQL Server 8.0+, MariaDB(ultima estable), PM2 (última estable), Nginx 1.18+ (producción), Git 2.x, Flutter SDK 3.11.1+ (app móvil), Android Studio (último estable).
- c. Permisos: Usuario de aplicación MySQL creado (no usar root en producción) con permisos mínimos sobre la base de datos `node_project_api`. Acceso SSH configurado al servidor.
- d. Acceso: El firewall debe permitir tráfico HTTP/HTTPS (puertos 80/443), puerto 3001 para la API Express.js. El puerto 3306 (MySQL) debe estar restringido solo a acceso local/interno.

2. Checklist de Preparación

- a. El servidor está encendido y accesible mariaDB(Ubuntu 22.04 LTS).
- b. Node.js v18+ está instalado y verificado (`node --version`).
- c. MySQL Server 8.0+ está instalado y corriendo (`systemctl status mysql`).
- d. Verificar que MariaDB esta instalado y ejecutandose.(`sudo systemctl status mariadb`)

- e. El usuario de aplicación MySQL existe y tiene permisos mínimos sobre node_project_api.
- f. Se realizó un backup completo de la base de datos y la aplicación actual.

3. Procedimiento de Instalación (Paso a Paso)

a. Verificar que se este compartiendo el puerto de la maquina virtual

- i. **Ir a confiiguracion de la maquina virtual, red y activar el puerto nat.**

b. Desplegar la base de datos:

- i. Iniciar sesion en mariaDB: `sudo mysql -u root -p`
- ii. Crear la base de datos: `CREATE DATABASE IF NOT EXISTS node_project_api;`
- iii. Cambiar el puerto en index.js del backend por el puerto 3000
- iv. Inicializar la api en visual studio code con el comando “npm run dev”, la carpeta del backend tiene por nombre Node_api asegurese de estar ejecutando el comando dentro de esa carpeta para eso vease el siguiente paso.

c. Desplegar el API Backend:

- i. Clonar repositorio, navegar a api/Node_api/ y ejecutar `npm install --production`.
- ii. RECOMENDACIÓN: Si tiene problemas al clonar el repositorio o el proyecto falla, descargue la carpeta .zip desde el mismo repositorio en github

d. Configurar entorno:

- i. En visual studio code descargar la extension de flutter, crear la carpeta de instalacion de flutter previamente en donde desee
- ii. Despues de la instalacion ejecutar el comando `flutter doctor` y verificar que cumple todos los requisitos(todas las casillas en verde)
- iii. Ddescargar android studio, en la instalacion asegurarse de descargar android sdk

- iv. Si el ckecklist ya esta todo en verde continuar con la ejecucion del proyecto, ir a la carpeta en consola del front en mobile “cd front-end-mobile/application_login”
- v. Antes de la ejecucion se debe verificar en el archivo front-end-mobile/application_login/lib/services/api_service.dart que la variable constante en la linea 8 (baseUrl) sea igual a la de axios en front-end-web/finance-frontend-react/src/api/axios.config.js con la direccion ip de la maquina para verificar ejecutar en la terminal del sistema el comando “ip a” y copiar la ipnet, pegarla en static const String _baseUrl = 'http://(DIRECCION IPNET):3000/api/v1'; sin los paréntesis.
- vi. Preparar el dispositivo Android para la ejecución, en los ajustes del dispositivo buscar numero de compilación en información del sistema, oprimir hasta que salga el mensaje “ya eres programador” esto activara las opciones del programador, dentro de estas buscaras la opción de depuración usb y activarla.
- vii. Para la ejecución en móvil, conectar el dispositivo al computador y ejecutar el comando flutter run en “cd front-end-mobile/application_login” comenzara la instalacion en el dispositivo movil
- e. Configurar variables de entorno (.env):
 - i. Crear archivo .env en api/Node_api/ con: SERVER_PORT, DB_NAME=node_project_api, DB_HOST, DB_USER, DB_PASSWORD, DB_PORT=3306, DB_DIALECT=mysql, JWT_SECRET.
 - ii. Ejecutar con modelsApp(true) en index.js para sincronizar tablas en el primer arranque, luego cambiar a modelsApp(false).
 - iii. En la carpeta afipMobile\front-end-web\finance-frontend-react\src\api ubicar axios y cambiar la direccion base url por la direccion network al ejecutar el comando npm run dev -- --host

asegurarse de solo copiar la direccion ip, deja el mismo puerto 3000.

- iv. Ejecutar el frontend con el comando `npm run dev -- --host`
Seleccionar la direccion ip de Network y esa sera la ip que se puede compartir, poner esa ip en `axios.js`

4. Verificación

- a. Verificar que la API responde: `curl http://localhost:3000/api/v1/user`
→ debe retornar JSON con error 401.
- b. La página de login del frontend web carga correctamente en `https://[dominio-produccion]`.
- c. Registrar un usuario de prueba y verificar que el login retorna token JWT y redirige al dashboard.
- d. Crear una cuenta financiera, registrar un ingreso y un gasto. Verificar que los saldos se actualizan correctamente en la BD de producción.
- e. Verificar que PM2 muestra estado "online" para `afip-api` y que los logs no presentan errores críticos.

5. Plan de Rollback

Si la verificación falla después del despliegue:

- a. Detener la API: `pm2 stop afip-api`.
- b. Restaurar la base de datos desde el backup previo al despliegue:
`mysql -u root -p node_project_api < backup_pre_deploy.sql`
- c. Revertir el código a la versión anterior: `git checkout <version-anterior>`, reinstalar dependencias (`npm install --production`) y reiniciar: `pm2 restart afip-api`.
- d. Notificar al equipo del fallo y comenzar investigación.

Tiempo Estimado de Rollback: 30 minutos.

6. Responsables:

- a. Ejecuta despliegue: Backend Developer / Administrador del Sistema
- b. Supervisa y verifica: Líder de QA / Tester